

ISIT307 - WEB SERVER PROGRAMMING

LECTURE 5.1 – PHP: OBJECT-ORIENTED PROGRAMMING – PART I



LECTURE PLAN

- Object-oriented programming concepts
- Class definition
- Use objects in PHP scripts

OBJECT-ORIENTED PROGRAMMING – KEY CONCEPTS

- object,
- encapsulation,
- association,
- aggregation,
- delegation,
- composition,
- dynamic binding,
- polymorphism,
- inheritance,
- hierarchical objects,
- abstract classes

INTRODUCTION

- **Object-oriented programming (OOP)** refers the concept of merging related variables and functions into a single interface
- An **object** refers to programming code and data that can be treated as an individual unit or component
- Object orientedness can be considered as co-operative problem solving through objects communicating with one another.

CLASSES

- A class is a blueprint, template that defines structure and behaviour of the objects
 - Help make complex programs easier to manage
 - Hide information that users of the object do not need to access or know about
 - Make it easier to reuse the code
- Properties - variables that represent the data or state of the object (instantiated from that class)
- Methods - functions that represent the behavior of objects and operate on the properties

CLASSES

- To create a class in PHP, the `class` keyword is used in a class definition

```
class ClassName {  
    properties  
    methods  
}
```

- Class names can begin with an uppercase letter to distinguish them from other variables
- Good programming practice is to store the classes in separate file and include in the code using `require()` or `require_once()`

CLASSES

- **Access specifiers/visibility modifier** control the access to individual properties and methods
- The access specifier must be included at the beginning of a property/method declaration
- There are three levels
 - public, private, and protected

CLASSES

- **public** – property/method can be access from anywhere, inside or outside the class
- **private** - property/method can be access only from within the class
- **protected** - property/method can be access from within the class and child classes
- In general, all properties should be private or protected
- All methods that needs to be accessed outside the class should be declared as public

CLASSES

- `$this` is a special variable in PHP that refers to the object itself (for which the property/method is executed)
- `$this` – is used inside the methods of a class to access its properties and methods

`$this->property`

`$this->method()`

OBJECTS

- Object is an **instance** of a class
 - using a class is to create an object from it
- The declaration of an object is by using the **new** operator
\$objectName = new ClassName();
- The -> (member selection notation) operator is used to access the properties and methods of an object

CLASSES & OBJECTS

- Some useful functions:
 - `get_class()` - retrieves the name of the class
 - `class_exists()` - determines if a class exists
 - `instanceof` operator - determines whether an object is instantiated from a given class

```
$myObj = new MyClass();  
echo 'The $myObj object is from' .  
    get_class($myObj) . " class.</p>";  
---  
if ($myObj instanceof MyClass) {...}  
---  
if (class_exists("MyClass")) {...}
```

CLASSES & OBJECTS

EXAMPLE

```
class BankAccount {
    public float $balance = 120.00;
    public function withdrawal(float $Amount):void {
        $this->balance -= $Amount;
    }
}

if (class_exists("BankAccount"))
    $checking = new BankAccount();
else
    exit("<p>The BankAccount class is not available!</p>");

echo "<p>Your checking account balance is \${$checking->balance}</p>";
$cash = 200;
$checking->withdrawal($cash);
echo "<p>After withdrawing \${$cash}, your checking account balance is
                                         \${$checking->balance}</p>";
```

CLASSES & OBJECTS

- A **constructor** is a special function that is called automatically when an object is instantiated (created)
 - it can have input arguments, or optional input arguments
- The constructor cannot be defined with the same name as the class (it will not be recognized as constructor in PHP8)

```
class BankAccount{  
    private float $balance;  
    public function __construct(float $balance) {  
        $this->balance = $balance;  
    }  
}
```

CLASSES & OBJECTS

- PHP 8.0 offers Constructor Property Promotion
 - simplifies the process of defining and initializing properties
 - allows property declaration and initialization directly in constructor parameters

CLASSES & OBJECTS

EXAMPLE

- Constructor Property Promotion

```
class BankAccount{  
    private float $balance;  
    public function __construct(float $balance) {  
        $this->balance = $balance;  
    }  
}
```

— — —

```
class BankAccount {  
    public function __construct(private float $balance) {}  
}
```

— — —

```
class BankAccount {  
    private string $name;  
    public function __construct(private float $balance, string $name) {  
        $this->name = $name; }  
}
```

CLASSES & OBJECTS

- **destructor** function is called when the object is destroyed
 - free up any resources allocated to an object after the object is destroyed
- destructor is commonly called when
 - a script ends
 - an object is manually deleted with `unset()`

```
function __destruct() {...}
```


CLASSES & OBJECTS

- Accessor/Mutator functions (get/set methods) are public methods (member functions) that can be called to retrieve or modify the value of a property (data member)
- Set functions modify data member values (mutator)
- Get functions retrieve data member values (accessor)

CLASSES & OBJECTS

EXAMPLE

```
class BankAccount {
    private float $balance=0;
    function __construct(float $bal=0)
    {
        $this->balance = $bal;
    }
    public function setBalance(float $newValue)
    {
        if ($newValue >0)
            $this->balance = $newValue;
    }
    public function getBalance():float
    {
        return $this->balance;
    }
}

$checking = new BankAccount();
$checking->setBalance(100);
echo "<p>Your checking account balance is " . $checking->getBalance()
    . "</p>\n";

$newCh = new BankAccount();
$newCh->setBalance(-50);
echo "<p>Your checking account balance is " . $newCh->getBalance()
    . "</p>\n";
```

CLASSES & OBJECTS

- The magic functions can also be used:
 - `__set()` – called when writing/setting data to protected or private property
 - `__get()` – called when accessing the protected or private property

```
class MyClass{
    private int $myP;
    function __get($name)
    {
        return $this->$name;
    }
    function __set($name, $value)
    {
        $this->$name = $value;
    }
}
```

```
$myV = new MyClass();
$myV->myP = 5;
echo $myV->myP;
```

